

Vulkan Notes

Texturing

Adam HAMOUCH

June 2026

1. Vue d'ensemble — les 4 étapes

Ajouter une texture en Vulkan requiert quatre étapes distinctes, chacune correspondant à un objet GPU différent :

1. **VkImage** — l'image stockée en mémoire GPU (les pixels)
2. **Upload** — copier les pixels depuis le CPU vers la **VkImage** via un staging buffer
3. **VkImageView** — une vue sur l'image (sous-ressource, format, type)
4. **VkSampler** — les règles de lecture (filtering, wrapping, anisotropie)

Ces quatre objets sont ensuite exposés au shader via un descriptor de type **COMBINED_IMAGE_SAMPLER**.

2. Chargement des pixels (stb_image)

stb_image est une lib header-only qui décompresse les formats courants (JPG, PNG, BMP...) en un tableau de bytes contigus.

```
#define STB_IMAGE_IMPLEMENTATION
#include <stb_image/stb_image.h>

int texWidth, texHeight, texChannels;
stbi_uc* pixels = stbi_load("texture.jpg",
    &texWidth, &texHeight, &texChannels, STBI_rgb_alpha);
// STBI_rgb_alpha force 4 canaux (R8G8B8A8) même si le fichier est RGB

VkDeviceSize imageSize = texWidth * texHeight * 4; // 4 octets par pixel

if (!pixels)
    throw std::runtime_error("échec du chargement de l'image!");

// ... utiliser pixels ...
stbi_image_free(pixels); // libérer après copie dans le staging buffer
```

3. Upload : staging buffer → VkImage

Comme pour les vertex buffers, on ne peut pas écrire directement en VRAM depuis le CPU. Le flow est :

1. Créer un staging buffer `HOST_VISIBLE` et y copier les pixels
2. Créer la `VkImage` finale en `DEVICE_LOCAL`
3. Transitionner le layout de l'image vers `TRANSFER_DST_OPTIMAL`
4. Copier le staging buffer vers l'image avec `vkCmdCopyBufferToImage`
5. Transitionner le layout vers `SHADER_READ_ONLY_OPTIMAL`
6. Detruire le staging buffer

```
// Staging buffer : CPU peut écrire
VkBuffer stagingBuffer; VkDeviceMemory stagingMem;
createBuffer(imageSize, VK_BUFFER_USAGE_TRANSFER_SRC_BIT,
    VK_MEMORY_PROPERTY_HOST_VISIBLE_BIT | VK_MEMORY_PROPERTY_HOST_COHERENT_BIT,
    stagingBuffer, stagingMem);

void* data;
vkMapMemory(device, stagingMem, 0, imageSize, 0, &data);
memcpy(data, pixels, imageSize);
vkUnmapMemory(device, stagingMem);
stbi_image_free(pixels);

// Image GPU finale
createImage(texWidth, texHeight, VK_FORMAT_R8G8B8A8_SRGB,
    VK_IMAGE_TILING_OPTIMAL,
    VK_IMAGE_USAGE_TRANSFER_DST_BIT | VK_IMAGE_USAGE_SAMPLED_BIT,
    VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT,
    textureImage, textureImageMemory);

transitionImageLayout(textureImage, VK_FORMAT_R8G8B8A8_SRGB,
    VK_IMAGE_LAYOUT_UNDEFINED, VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL);

copyBufferToImage(stagingBuffer, textureImage, texWidth, texHeight);

transitionImageLayout(textureImage, VK_FORMAT_R8G8B8A8_SRGB,
    VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL,
    VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL);

vkDestroyBuffer(device, stagingBuffer, nullptr);
vkFreeMemory(device, stagingMem, nullptr);
```

4. VkImage — creation

```
void createImage(uint32_t w, uint32_t h, VkFormat format,
    VkImageTiling tiling, VkImageUsageFlags usage,
    VkMemoryPropertyFlags properties,
    VkImage& image, VkDeviceMemory& imageMemory)
{
    VkImageCreateInfo imageInfo{};
    imageInfo.sType = VK_STRUCTURE_TYPE_IMAGE_CREATE_INFO;
    imageInfo.imageType = VK_IMAGE_TYPE_2D;
    imageInfo.extent = {w, h, 1};
    imageInfo.mipLevels = 1;
    imageInfo.arrayLayers = 1;
    imageInfo.format = format;
    imageInfo.tiling = tiling; // OPTIMAL = GPU choisit le layout interne
    imageInfo.initialLayout = VK_IMAGE_LAYOUT_UNDEFINED;
    imageInfo.usage = usage;
    imageInfo.samples = VK_SAMPLE_COUNT_1_BIT;
    imageInfo.sharingMode = VK_SHARING_MODE_EXCLUSIVE;
    vkCreateImage(device, &imageInfo, nullptr, &image);

    VkMemoryRequirements memReq;
    vkGetImageMemoryRequirements(device, image, &memReq);
    // ... vkAllocateMemory + vkBindImageMemory ...
}
```

Tiling :

- **LINEAR** : pixels en row-major en memoire, accessible par le CPU. Peu performant pour le GPU.
- **OPTIMAL** : layout interne opaque choisi par le GPU pour maximiser les performances. Obligatoire pour les textures de rendu.

5. Image Layouts et Transitions

Une image Vulkan est toujours dans un **layout** qui determine comment ses donnees sont organisees en memoire interne du GPU. Changer de layout necessite une **pipeline barrier** explicite.

Layout	Usage
UNDEFINED	Etat initial, contenu inconnu
TRANSFER_DST_OPTIMAL	Destination d'une copie (upload)
TRANSFER_SRC_OPTIMAL	Source d'une copie
SHADER_READ_ONLY_OPTIMAL	Lecture par un shader
COLOR_ATTACHMENT_OPTIMAL	Rendu vers cette image
PRESENT_SRC_KHR	Presentation a l'ecran
GENERAL	Universel mais non optimise

Pipeline barrier — mecanisme de transition de layout et de synchronisation memoire :

```
void transitionImageLayout(VkImage image, VkFormat format,
    VkImageLayout oldLayout, VkImageLayout newLayout)
{
    VkCommandBuffer cmd = beginSingleTimeCommands();

    VkImageMemoryBarrier barrier{};
    barrier.sType          = VK_STRUCTURE_TYPE_IMAGE_MEMORY_BARRIER;
    barrier.oldLayout       = oldLayout;
    barrier.newLayout       = newLayout;
    barrier.srcQueueFamilyIndex = VK_QUEUE_FAMILY_IGNORED;
    barrier.dstQueueFamilyIndex = VK_QUEUE_FAMILY_IGNORED;
    barrier.image           = image;
    barrier.subresourceRange = {VK_IMAGE_ASPECT_COLOR_BIT, 0, 1, 0, 1};

    VkPipelineStageFlags srcStage, dstStage;

    if (oldLayout == VK_IMAGE_LAYOUT_UNDEFINED &&
        newLayout == VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL) {
        barrier.srcAccessMask = 0;
        barrier.dstAccessMask = VK_ACCESS_TRANSFER_WRITE_BIT;
        srcStage = VK_PIPELINE_STAGE_TOP_OF_PIPE_BIT;
        dstStage = VK_PIPELINE_STAGE_TRANSFER_BIT;
    }
    else if (oldLayout == VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL &&
        newLayout == VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL) {
        barrier.srcAccessMask = VK_ACCESS_TRANSFER_WRITE_BIT;
        barrier.dstAccessMask = VK_ACCESS_SHADER_READ_BIT;
        srcStage = VK_PIPELINE_STAGE_TRANSFER_BIT;
        dstStage = VK_PIPELINE_STAGE_FRAGMENT_SHADER_BIT;
    }

    vkCmdPipelineBarrier(cmd, srcStage, dstStage,
        0, 0, nullptr, 0, nullptr, 1, &barrier);

    endSingleTimeCommands(cmd);
}
```

srcAccessMask / dstAccessMask : quelles operations memoire doivent etre terminees avant/apres la barrier.

srcStage / dstStage : a quel moment du pipeline la barrier s'insere.

TOP_OF_PIPE_BIT : pseudo-stage "avant tout" — utile quand l'operation ne depend de rien.

6. copyBufferToImage

```
void copyBufferToImage(VkBuffer buffer, VkImage image,
    uint32_t width, uint32_t height)
{
    VkCommandBuffer cmd = beginSingleTimeCommands();

    VkBufferImageCopy region{};
    region.bufferOffset    = 0;
    region.bufferRowLength = 0; // 0 = tightly packed
    region.bufferImageHeight = 0;
    region.imageSubresource = {VK_IMAGE_ASPECT_COLOR_BIT, 0, 0, 1};
    region.imageOffset      = {0, 0, 0};
    region.imageExtent      = {width, height, 1};

    vkCmdCopyBufferToImage(cmd, buffer, image,
        VK_IMAGE_LAYOUT_TRANSFER_DST_OPTIMAL, 1, &region);

    endSingleTimeCommands(cmd);
}
```

L'image doit etre en **TRANSFER_DST_OPTIMAL** avant l'appel. **bufferRowLength = 0** signifie que les pixels sont continus en memoire (pas de padding entre les lignes).

7. VkImageView

Une **image view** est une fenetre sur une image — elle decrit quelle partie de l'image est accessible et comment l'interpreter. On ne bind jamais une `VkImage` directement au shader, toujours via une view.

```
VkImageView createImageView(VkImage image, VkFormat format)
{
    VkImageViewCreateInfo viewInfo{};
    viewInfo.sType = VK_STRUCTURE_TYPE_IMAGE_VIEW_CREATE_INFO;
    viewInfo.image = image;
    viewInfo.viewType = VK_IMAGE_VIEW_TYPE_2D;
    viewInfo.format = format;
    viewInfo.subresourceRange = {VK_IMAGE_ASPECT_COLOR_BIT, 0, 1, 0, 1};
    // baseMipLevel=0, levelCount=1, baseArrayLayer=0, layerCount=1

    VkImageView imageView;
    vkCreateImageView(device, &viewInfo, nullptr, &imageView);
    return imageView;
}
```

La meme fonction sert pour les image views de la swap chain et pour les textures — c'est pourquoi on la refactorise en helper.

8. VkSampler

Le sampler decrit **comment** lire la texture, independamment de la texture elle-meme. Un seul sampler peut etre reutilise avec plusieurs textures.

```
VkSamplerCreateInfo samplerInfo{};
samplerInfo.sType = VK_STRUCTURE_TYPE_SAMPLER_CREATE_INFO;
samplerInfo.magFilter = VK_FILTER_LINEAR; // zoom in : interpolation lineaire
samplerInfo.minFilter = VK_FILTER_LINEAR; // zoom out : interpolation lineaire
samplerInfo.addressModeU = VK_SAMPLER_ADDRESS_MODE_REPEAT;
samplerInfo.addressModeV = VK_SAMPLER_ADDRESS_MODE_REPEAT;
samplerInfo.addressModeW = VK_SAMPLER_ADDRESS_MODE_REPEAT;
samplerInfo.anisotropyEnable = VK_TRUE;
samplerInfo.maxAnisotropy = 16.0f;
samplerInfo.borderColor = VK_BORDER_COLOR_INT_OPAQUE_BLACK;
samplerInfo.unnormalizedCoordinates = VK_FALSE; // UV entre [0,1]
samplerInfo.compareEnable = VK_FALSE; // utile pour shadow maps
samplerInfo.mipmapMode = VK_SAMPLER_MIPMAP_MODE_LINEAR;
samplerInfo.minLod = 0.0f;
samplerInfo.maxLod = 0.0f; // 0 = pas de mipmaps
vkCreateSampler(device, &samplerInfo, nullptr, &textureSampler);
```

Address modes (UV hors [0,1]) :

Mode	Comportement
REPEAT	La texture se repete en tile
MIRRORED_REPEAT	Repete en miroir
CLAMP_TO_EDGE	Etire le pixel du bord
CLAMP_TO_BORDER	Couleur unie (borderColor)

Anisotropie : ameliore la qualite des textures vues en angle rasant. Necessite `samplerAnisotropy = VK_TRUE` dans les features du device physique.

9. Descriptor Layout & Sets pour la texture

La texture est exposee au fragment shader via un descriptor de type `COMBINED_IMAGE_SAMPLER` — image view et sampler fusionnees en un seul binding.

```
// Layout : binding 0 = UBO, binding 1 = sampler
VkDescriptorSetLayoutBinding uboBinding{};
uboBinding.binding           = 0;
uboBinding.descriptorType    = VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER;
uboBinding.stageFlags        = VK_SHADER_STAGE_VERTEX_BIT;

VkDescriptorSetLayoutBinding samplerBinding{};
samplerBinding.binding       = 1;
samplerBinding.descriptorType = VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER;
samplerBinding.descriptorCount = 1;
samplerBinding.stageFlags     = VK_SHADER_STAGE_FRAGMENT_BIT;

// Pool : deux types de descripteurs
std::array<VkDescriptorPoolSize, 2> poolSizes{};
poolSizes[0] = {VK_DESCRIPTOR_TYPE_UNIFORM_BUFFER,      N};
poolSizes[1] = {VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER, N};

// Ecriture du descriptor set
VkDescriptorImageInfo imageInfo{};
imageInfo.imageLayout = VK_IMAGE_LAYOUT_SHADER_READ_ONLY_OPTIMAL;
imageInfo.imageView   = textureImageView;
imageInfo.sampler      = textureSampler;

descriptorWrites[1].dstBinding      = 1;
descriptorWrites[1].descriptorType = VK_DESCRIPTOR_TYPE_COMBINED_IMAGE_SAMPLER;
descriptorWrites[1].pImageInfo      = &imageInfo;
```

10. Coordonnees UV & Shaders

Les coordonnees UV doivent etre ajoutees au struct `Vertex` et declarees dans les shaders :

```
struct Vertex {
    Vector2D pos;
    Vector3D color;
    Vector2D texCoord; // UV entre [0,1]

    static std::array<VkVertexInputAttributeDescription, 3> getAttributeDescriptions() {
        // location 0 : pos, location 1 : color, location 2 : texCoord
        attributeDescriptions[2].location = 2;
        attributeDescriptions[2].format   = VK_FORMAT_R32G32_SFLOAT;
        attributeDescriptions[2].offset   = offsetof(Vertex, texCoord);
    }
};

// Vertex shader
layout(location = 2) in vec2 inTexCoord;
layout(location = 1) out vec2 fragTexCoord; // passe au frag shader

// Fragment shader
layout(binding = 1) uniform sampler2D texSampler;
layout(location = 1) in vec2 fragTexCoord;
void main() {
    outColor = texture(texSampler, fragTexCoord);
}
```

11. Helpers single-time commands

Pattern pour executer des operations GPU ponctuelles (upload, transition) hors de la boucle de rendu :

```
VkCommandBuffer beginSingleTimeCommands() {  
    // Alloue un CB temporaire avec ONE_TIME_SUBMIT_BIT  
}  
  
void endSingleTimeCommands(VkCommandBuffer cmd) {  
    vkEndCommandBuffer(cmd);  
    // Submit + vkQueueWaitIdle (bloque le CPU)  
    vkFreeCommandBuffers(device, commandPool, 1, &cmd);  
}
```

Limite : `vkQueueWaitIdle` bloque le CPU a chaque operation. En production, on batche toutes les operations d'upload dans un seul command buffer soumis une fois. Pour les textures en cours de jeu, on utilise des fences plutot que `WaitIdle`.

Pieges frequents :

- Appeler `transitionImageLayout` deux fois vers le meme layout → erreur de validation (l'image est deja dans ce layout).
- `vkCmdCopyBufferToImage` apres `endSingleTimeCommands` → le CB est deja soumis et libere.
- Oublier `VK_IMAGE_USAGE_SAMPLED_BIT` dans `createImage` → le shader ne peut pas lire l'image.
- Array de 2 attribute descriptions pour 3 attributs → out of bounds silencieux sous MSVC.
- `anisotropyEnable` sans activer la feature dans `deviceFeatures.samplerAnisotropy` → crash ou comportement indefini.